

Swagger UI

Auth Service · Bite&Go

Guía de uso + referencia completa de endpoints

Versión 1.0 · 19 de April de 2026

Servicio: **auth-service-bite-go**

Base URL: <http://localhost:3000/swagger>

Contenido

PARTE 1 · Guía de uso

1. ¿Qué es Swagger?	3
2. Cómo acceder a Swagger UI	3
3. Anatomía de la interfaz	4
4. Flujo típico de prueba (login + endpoint protegido)	5
5. Interpretar las respuestas	6
6. Exportar la colección	7
7. Solución de problemas comunes	7

PARTE 2 · Referencia de endpoints

8. Convenciones generales	8
9. Autenticación	8
10. POST /api/v1/auth/register	9
11. POST /api/v1/auth/login	11
12. GET /api/v1/auth/profile	12
13. POST /api/v1/auth/profile/by-id	13
14. POST /api/v1/auth/verify-email	14
15. POST /api/v1/auth/resend-verification	15
16. POST /api/v1/auth/forgot-password	16
17. POST /api/v1/auth/reset-password	17
18. GET /health	18
19. Esquemas de datos (DTOs)	19
20. Códigos de estado y rate limiting	20

PARTE 1

Guía de uso de Swagger UI

1. ¿Qué es Swagger?

Swagger UI es una interfaz web autogenerada que documenta todos los endpoints REST del auth-service. Se construye automáticamente a partir de los atributos de los controladores .NET y los *XML comments* del código, por lo que está **siempre sincronizada con la implementación real**: si un endpoint cambia, la documentación cambia al reiniciar el servicio.

Además de mostrar la documentación, Swagger permite **ejecutar peticiones** directamente desde el navegador, enviar tokens JWT, subir archivos y ver las respuestas con su código de estado, headers y body. Es el reemplazo cómodo de Postman durante desarrollo.

2. Cómo acceder a Swagger UI

Con el servicio corriendo (Docker Compose o `dotnet run`), abre en el navegador:

```
http://localhost:3000/swagger
```

Swagger UI está habilitada en dos casos:

- Cuando el entorno es **Development** (por defecto en `appsettings.json`).
- Cuando la variable de entorno `ENABLE_SWAGGER=true` está establecida (el Dockerfile ya lo hace).

En producción

Desactiva Swagger antes de desplegar a un entorno público para no exponer la superficie de API. Poner `ENABLE_SWAGGER=false` y usar `ASPNETCORE_ENVIRONMENT=Production` basta.

3. Anatomía de la interfaz

Al abrir Swagger UI verás tres zonas principales:

Encabezado	Título (Bite&Go - Auth Service API), versión y link al repositorio. Ahí también está el botón Authorize .
Lista de endpoints	Agrupados por controlador (en este caso <i>Auth</i>). Cada uno muestra el método HTTP coloreado, la ruta y un resumen corto.
Detalles del endpoint	Al hacer clic en una ruta se expande: descripción larga, parámetros, cuerpo de la petición, respuestas posibles y el botón Try it out .

4. Flujo típico de prueba

Este es el orden recomendado para probar que todo funciona end-to-end. Asume que el servicio está corriendo en `localhost:3000` y que la base de datos ya tiene los roles sembrados por el *DataSeeder*:

Paso 1 — Registrar un usuario

Expande **POST /api/v1/auth/register** y pulsa *Try it out*. Rellena los campos del formulario (son *multipart/form-data*, así que también puedes adjuntar una foto en *profilePicture*). Pulsa *Execute*. La respuesta 201 te confirmará el usuario creado y enviará un correo de verificación al email que pusiste.

Paso 2 — Verificar el email

Abre tu bandeja de entrada, copia el token del correo y úsalo en **POST /api/v1/auth/verify-email**. Si prefieres saltártelo en desarrollo, puedes marcar el usuario como verificado en la base de datos directamente (ver Parte 2).

Paso 3 — Login

Expande **POST /api/v1/auth/login**, usa tu *email o username* y contraseña. Copia el valor del campo token de la respuesta (es el JWT).

Paso 4 — Autorizar en Swagger

Pulsa el botón **Authorize** del encabezado. Pega el token (sin la palabra 'Bearer', Swagger la añade sola) y pulsa *Authorize*. A partir de ese momento todas las peticiones a endpoints protegidos incluyen el header `Authorization: Bearer ...`

Paso 5 — Probar un endpoint protegido

Expande **GET /api/v1/auth/profile** y ejecuta. Debería devolver 200 con tus datos. Si devuelve 401, revisa que el token no esté expirado.

5. Interpretar las respuestas

Tras ejecutar una petición, Swagger muestra tres bloques útiles:

- **Curl**: el comando equivalente que podrías pegar en la terminal. Útil para reproducir el caso.
- **Request URL**: la URL final exacta que se llamó.
- **Server response**: código de estado, body y headers. Los códigos siguen la convención estándar HTTP.

6. Exportar la colección

Si tu equipo usa Postman, Insomnia o Bruno, puedes importar el spec OpenAPI directamente desde:

```
http://localhost:3000/swagger/v1/swagger.json
```

Desde Postman: *Import* → *Link* y pega esa URL. Quedan los 9 endpoints organizados con sus schemas ya definidos.

7. Solución de problemas comunes

La página /swagger devuelve 404	Swagger no está habilitado. Verifica que <code>ENABLE_SWAGGER=true</code> o que el entorno sea <code>Development</code> .
Error 401 en endpoints protegidos	Token vencido o no enviado. Expira a las 120 minutos. Haz login de nuevo y repite el paso <i>Authorize</i> .
Error 429 Too Many Requests	El rate limit se activó. Hay dos políticas: <i>AuthPolicy</i> (endpoints de registro/login) y <i>ApiPolicy</i> (otros). Espera unos segundos.
El avatar no se sube	En <i>register</i> , el campo es <i>profilePicture</i> y acepta JPG/PNG hasta 10 MB. Si supera el límite devuelve 413.
CORS error desde el frontend	El origen de tu frontend debe estar en <code>Security.AllowedOrigins</code> en <code>appsettings.json</code> . Por defecto ya incluye 5173 y 3000-3002.

PARTE 2

Referencia de endpoints

8. Convenciones generales

Todos los endpoints del servicio comparten estas reglas:

Base URL	http://localhost:3000 (desarrollo Docker)
Prefijo	/api/v1/auth para todos los endpoints funcionales
Formato	JSON en requests (salvo register, que es multipart/form-data)
Charset	UTF-8
Auth	JWT Bearer — solo si el endpoint lo requiere explícitamente
Rate limit	AuthPolicy (register/login/password) o ApiPolicy (lecturas)
Timezone	Todas las fechas en UTC (ISO 8601)

9. Autenticación

El servicio emite tokens **JWT HS256**. Para consumir endpoints protegidos, añade el header:

```
Authorization: Bearer <token>
```

Estructura del payload:

```
{
  "sub": "<userId>",
  "role": "Cliente | Mesero | Cocinero | Admin_Restaurante | SuperAdmin",
  "jti": "<guid único del token>",
  "iat": 1734567890,
  "exp": 1734575090,
  "iss": "BiteGoAuthService",
  "aud": "BiteGoServices"
}
```

Expiración

Los tokens duran 120 minutos por defecto (configurable en `JwtSettings.ExpiryInMinutes`). Pasado ese tiempo deberás volver a hacer login.

10. Registrar un usuario

POST**/api/v1/auth/register***Registra un nuevo usuario con opción de avatar*

Autenticación	Pública (no requiere token)
Rate limit	AuthPolicy

Content-Type	multipart/form-data
Max body size	10 MB (por el avatar)

Parámetros del formulario:

Campo	Tipo	Requerido	Restricciones
name	string	Sí	máx. 25 caracteres
surname	string	Sí	máx. 25 caracteres
username	string	Sí	único en el sistema
email	string	Sí	formato email válido, único
password	string	Sí	mínimo 8 caracteres
phone	string	Sí	exactamente 8 dígitos
profilePicture	file	No	JPG/PNG, sube a Cloudinary

Respuestas:

Código	Descripción	Cuerpo
201	Usuario creado y correo de verificación enviado	RegisterResponseDto con los datos del usuario y un flag emailVerificationRequired=true
400	Validación fallida	Lista de errores por campo
409	Email o username ya registrado	{ success: false, message: 'El usuario ya existe' }
413	Avatar excede 10 MB	—
429	Rate limit excedido	Retry-After header presente

Ejemplo con curl:

```
curl -X POST http://localhost:3000/api/v1/auth/register \
-F "name=Marcos" \
-F "surname=Bajera" \
-F "username=marcos" \
-F "email=marcos@demo.com" \
-F "password=SuperSeguro1!" \
-F "phone=55512345" \
-F "profilePicture=@avatar.jpg"
```

11. Iniciar sesión

POST**/api/v1/auth/login***Autentica un usuario y devuelve un JWT*

Autenticación	Pública
Rate limit	AuthPolicy
Content-Type	application/json

Cuerpo de la petición:

```
{
  "emailOrUsername": "marcos@demo.com",
  "password": "SuperSeguro1!"
}
```

Respuestas:

Código	Descripción	Cuerpo
200	Login exitoso	AuthResponseDto con token, expiresAt y userDetails
400	emailOrUsername o password vacíos	Lista de errores de validación
401	Credenciales inválidas	{ success: false, message: 'Credenciales incorrectas' }
403	Email no verificado	{ success: false, message: 'Email pendiente de verificación' }
429	Rate limit excedido	—

Ejemplo de respuesta 200:

```
{
  "success": true,
  "message": "Login exitoso",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "expiresAt": "2026-04-19T02:30:00Z",
  "userDetails": {
    "id": "...",
    "username": "marcos",
    "profilePicture": "https://res.cloudinary.com/.../avatar.jpg",
    "role": "Cliente"
  }
}
```

12. Obtener el perfil propio

GET**/api/v1/auth/profile***Devuelve el perfil del usuario autenticado*

Autenticación	JWT Bearer (requerido)
Rate limit	Global (sin política específica)

Parámetros	Ninguno — el ID se extrae del claim <i>sub</i> del token
------------	--

Respuestas:

Código	Descripción	Cuerpo
200	Perfil obtenido	{ success, message, data: UserResponseDto }
401	Sin token o token inválido	—
404	El usuario del token ya no existe (raro)	—

13. Obtener perfil por ID

POST**/api/v1/auth/profile/by-id***Consulta puntual de un usuario por su ID*

Autenticación	Pública (pero rate-limitada)
Rate limit	ApiPolicy
Content-Type	application/json

Cuerpo:

```
{ "userId": "12345" }
```

Respuestas:

Código	Descripción	Cuerpo
200	Perfil obtenido	{ success, message, data: UserResponseDto }
400	userId vacío	{ success: false, message: 'El userId es requerido' }
404	Usuario no encontrado	{ success: false, message: 'Usuario no encontrado' }
429	Rate limit excedido	—

14. Verificar email

POST**/api/v1/auth/verify-email***Valida el token recibido por correo tras el registro*

Autenticación	Pública
Rate limit	ApiPolicy
Content-Type	application/json

Cuerpo:

```
{ "token": "<token-recibido-por-correo>" }
```

Respuestas:

Código	Descripción	Cuerpo
200	Email verificado	EmailResponseDto con success=true
400	Token inválido o expirado	EmailResponseDto con success=false

15. Reenviar email de verificación

POST

/api/v1/auth/resent-verification

Reenvía el correo si el anterior se perdió

Autenticación	Pública
Rate limit	AuthPolicy
Content-Type	application/json

Cuerpo:

```
{ "email": "usuario@demo.com" }
```

Respuestas:

Código	Descripción	Cuerpo
200	Correo reenviado	EmailResponseDto con success=true
400	El email ya había sido verificado	{ success: false, message: 'El email ya ha sido verificado' }
404	Usuario no encontrado	—
503	Fallo del servidor SMTP	Revisar logs del auth-service

16. Olvidé mi contraseña

POST

/api/v1/auth/forgot-password

Envía un correo con un token de recuperación

Autenticación	Pública
Rate limit	AuthPolicy
Content-Type	application/json

Cuerpo:

```
{ "email": "usuario@demo.com" }
```

Seguridad

Este endpoint siempre responde 200, aunque el email no exista, para evitar que terceros descubran qué correos están registrados (enumeration attack).

Respuestas:

Código	Descripción	Cuerpo
200	Si el correo existe, se envió el enlace	EmailResponseDto con mensaje genérico
503	Fallo del servidor SMTP	—

17. Restablecer contraseña

POST

/api/v1/auth/reset-password*Cambia la contraseña usando el token de recuperación*

Autenticación	Pública
Rate limit	AuthPolicy
Content-Type	application/json

Cuerpo:

```
{
  "token": "<token-del-correo>",
  "newPassword": "NuevaPass2025!"
}
```

Respuestas:

Código	Descripción	Cuerpo
200	Contraseña actualizada	EmailResponseDto con success=true
400	Token inválido, expirado o password corta (<8 chars)	EmailResponseDto con success=false

18. Health check

GET

/health*Verifica que el servicio está vivo*

Autenticación	Pública
Rate limit	Ninguno
Alias	/api/v1/health (misma respuesta)

Respuesta 200:

```
{
  "status": "Healthy",
  "timestamp": "2026-04-19T02:30:00.000Z"
}
```

Este endpoint se usa en el *healthcheck* del contenedor Docker y puede apuntarse desde un balanceador o monitor externo (Uptime Kuma, PagerDuty, etc.).

19. Esquemas de datos (DTOs)

Referencia rápida de los objetos que viajan en los cuerpos de petición/respuesta.

UserResponseDto

Campo	Tipo	Siempre	Notas
id	string	Sí	GUID del usuario
name	string	Sí	Nombre
surname	string	Sí	Apellido
username	string	Sí	Login alternativo al email
email	string	Sí	
profilePicture	string	Sí	URL completa de Cloudinary; cadena vacía si no subió
phone	string	Sí	8 dígitos
role	string	Sí	Uno de los 5 roles Bite&Go;
status	bool	Sí	true = activo
isEmailVerified	bool	Sí	
createdAt	datetime	Sí	UTC
updatedAt	datetime	Sí	UTC

UserDetailsDto (dentro de AuthResponseDto)

Campo	Tipo	Siempre	Notas
id	string	Sí	
username	string	Sí	
profilePicture	string	Sí	URL del avatar
role	string	Sí	

AuthResponseDto (respuesta de login)

Campo	Tipo	Siempre	Notas
success	bool	Sí	
message	string	Sí	Mensaje de confirmación
token	string	Sí	JWT firmado con HS256
userDetails	UserDetailsDto	Sí	Datos mínimos del usuario
expiresAt	datetime	Sí	UTC — cuándo expira el token

EmailResponseDto (verify / resend / forgot / reset)

Campo	Tipo	Siempre	Notas
success	bool	Sí	
message	string	Sí	Texto localizado en español

Campo	Tipo	Siempre	Notas
data	object?	No	Carga adicional opcional (null normalmente)

20. Códigos de estado y rate limiting

Códigos de estado HTTP usados:

Código	Significado	Cuándo lo verás	
200 OK	Éxito genérico	Login, profile, verify-email, etc.	
201 Created	Recurso creado	Registro exitoso	
400 Bad Request	Validación fallida	Campos obligatorios vacíos, token vencido	
401 Unauthorized	Sin auth o auth inválida	Endpoints protegidos sin token	
403 Forbidden	Identidad válida pero sin permiso	Email no verificado en login	
404 Not Found	Recurso no existe	profile/by-id con ID inválido	
409 Conflict	Choque con estado actual	Registro con email/username ya usado	
413 Payload Too Large	Archivo demasiado grande	Avatar > 10 MB	
429 Too Many Requests	Rate limit excedido	Muchos intentos seguidos de login/register	
500 Internal Server Error	Error no controlado	Revisar logs del auth-service	
503 Service Unavailable	Dependencia externa caída	SMTP no responde	

Políticas de rate limiting:

Política	Aplica a	Límite	Ventana
AuthPolicy	register, login, resend, forgot, reset	5 req	1 minuto por IP
ApiPolicy	verify-email, profile/by-id	30 req	1 minuto por IP
Default	Resto de rutas	Sin rate limit específico	—

Cuando se excede el límite, la respuesta incluye el header `Retry-After` indicando cuántos segundos esperar.

Documento generado automáticamente a partir del código fuente del auth-service-bite-go. Si un endpoint cambia, regenera este PDF para mantenerlo sincronizado.